Distributed Computing Fundamentals

Why One Computer Stopped Being Enough

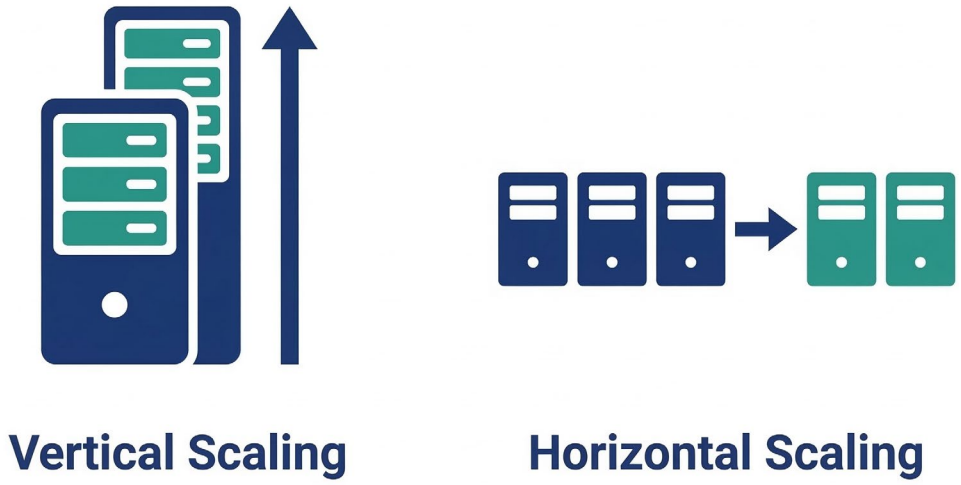
Callback to Week 4: Vertical vs. Horizontal Scaling

Vertical Scaling

- Get a bigger machine
- More CPU, RAM, storage
- Quickly gets expensive
- Hardware limits exist

Horizontal Scaling

- Add more machines
- Commodity hardware
- Potentially indefinite
- Requires coordination



In Week 4, these were concepts. This week, we cover the distinction.

You've built SQL queries, loaded data into Snowflake, run Glue jobs, and piped data through S3. Every one of those tools ran on someone else's really big computer.

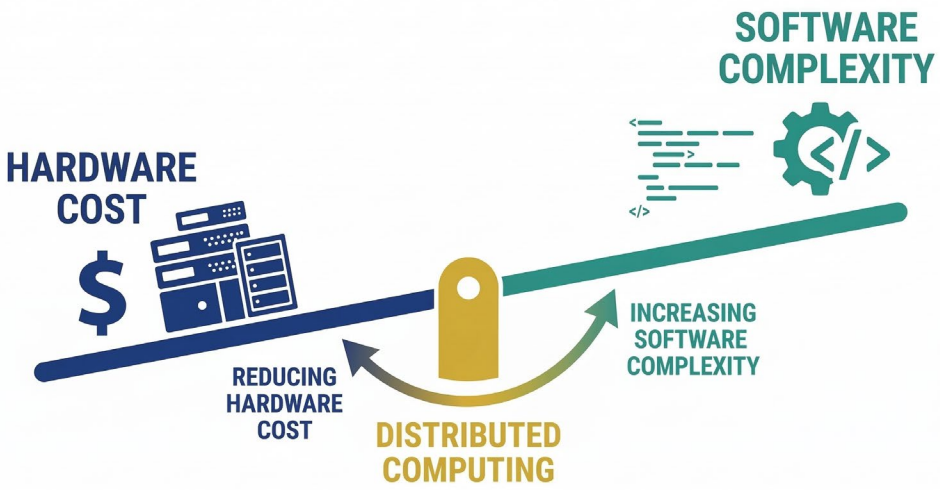
What happens when even a really big computer isn't big enough?

The Punchline Up Front

Distributed computing is not a technology choice. It's a concession.

Nobody wakes up and says 'I wish my computation were spread across 1,000 machines that might fail at any moment.' You do it because the alternative (waiting, or not doing the analysis at all) is worse.

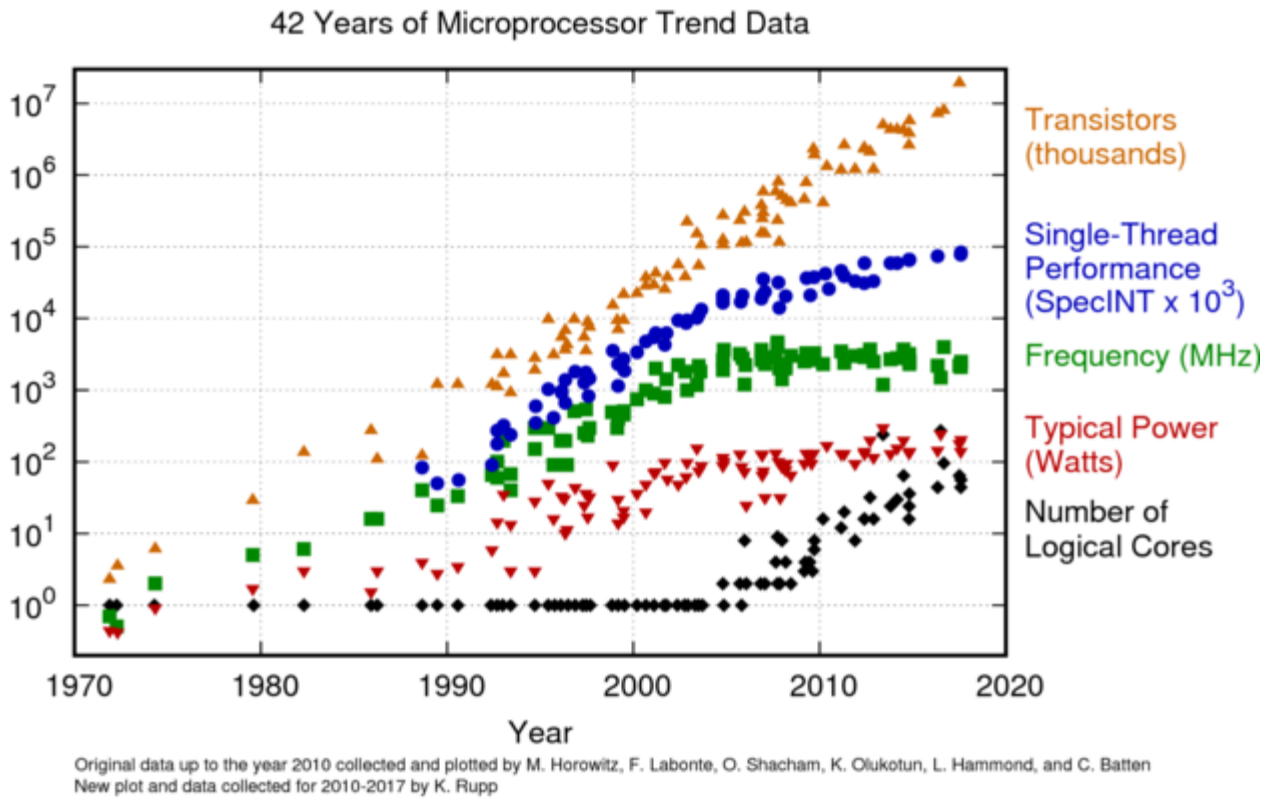
Every design decision in this space trades hardware cost for software complexity.



The Three Walls: Physics, Data, and Economics

Part 1 · Why Vertical Scaling Ran Out of Road

Wall #1: The Clock Speed Wall



For decades, software got faster for free: just run it on next year's chip. That strategy ended around 2004.

Why Clock Speeds Stopped Rising

Power Wall

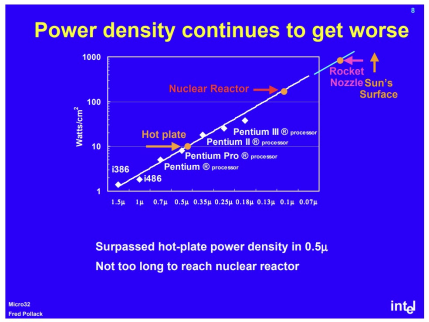
Power consumption scales roughly as the cube of frequency. At 4 GHz, chips approached the thermal density of a nuclear reactor surface.

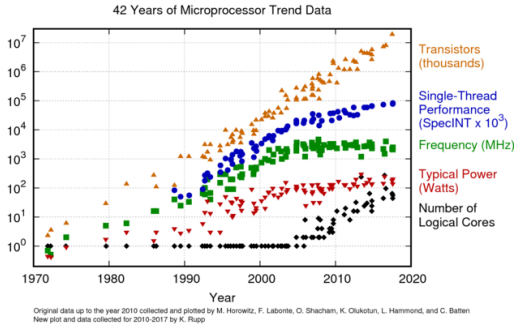
ILP Limits

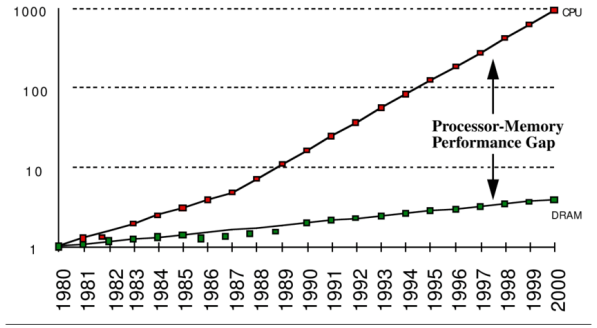
CPUs used tricks (pipelining, branch prediction, out-of-order execution) to go faster. By 2004, diminishing returns on all of them.

Memory Wall

Processors got faster than memory could feed them. CPUs spent more time waiting for data than computing.



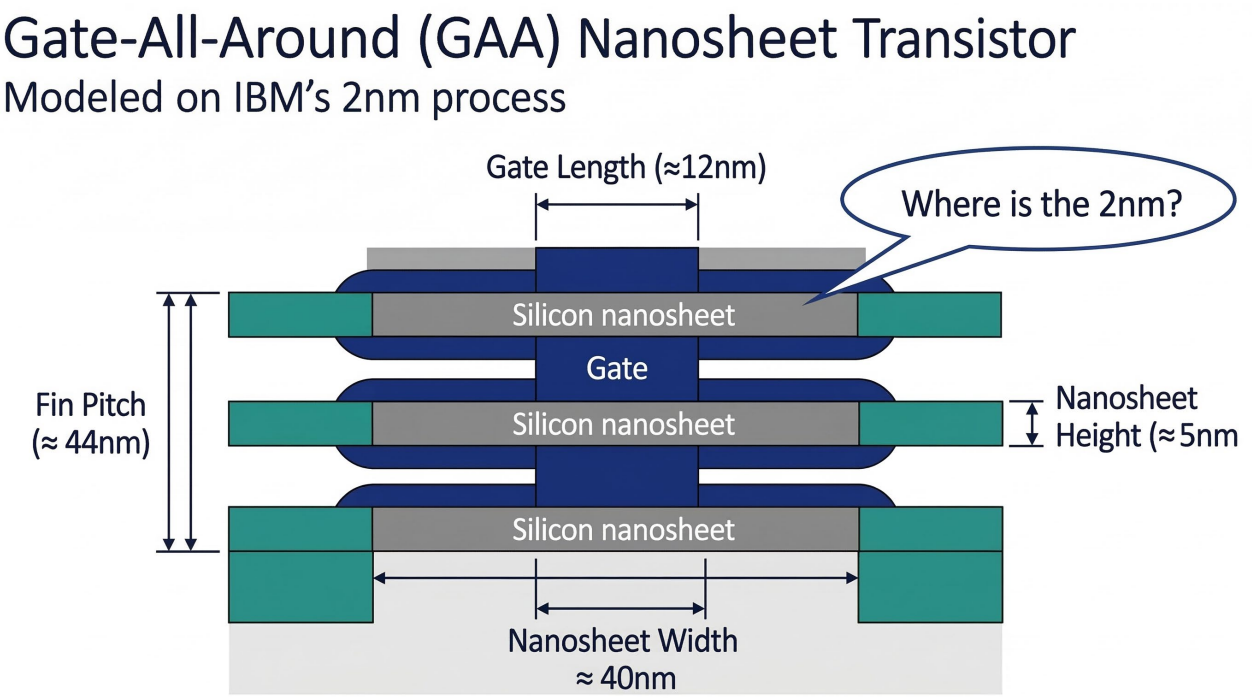




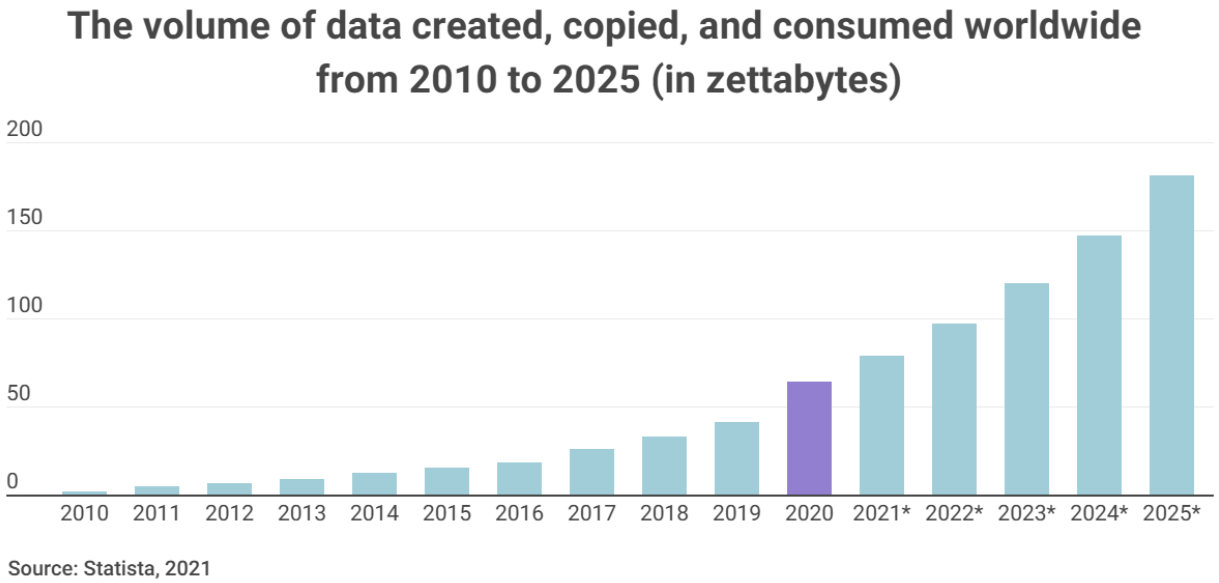
The industry's response: go multi-core. But multi-core only helps if software can exploit parallelism.

Aside: What "2nm" Actually Means

- Historically, process node = gate length
- That broke down ~2012 with FinFETs
- IBM's "2nm": gate ~12nm, pitch ~44nm, nanosheets ~40nm wide
- **Nothing on the chip is 2nm**
- The number is a marketing label
- Intel's "10nm" ≈ TSMC's "7nm"



Wall #2: The Data Wall



· Web indexing (Google's founding problem)
· Application log data (every click, session, error)
· Sensor data, SCADA, early IoT
· Digitization of gov't records, health, finance

The problem compounds: it's not just storing more data. It's querying, joining, and transforming it (operations that can be $O(n^2)$ or worse).

The Data Wall: Policy Context

Census Bureau
Tabulate every household. Apply differential privacy. Publish on deadline.

CMS
Billions of Medicare/Medicaid claims records.

IRS
Hundreds of millions of tax returns.

These agencies didn't choose 'big data.' The data arrived because of the missions they serve.

The Data Wall: Policy Context (cont.)

DoD / Intel

Satellite imagery, sensor networks, continental-scale data.

Cross-Agency

Joining IRS + CMS + SSA = joins across billions of rows.

These agencies didn't choose 'big data.' The data arrived because of the missions they serve.

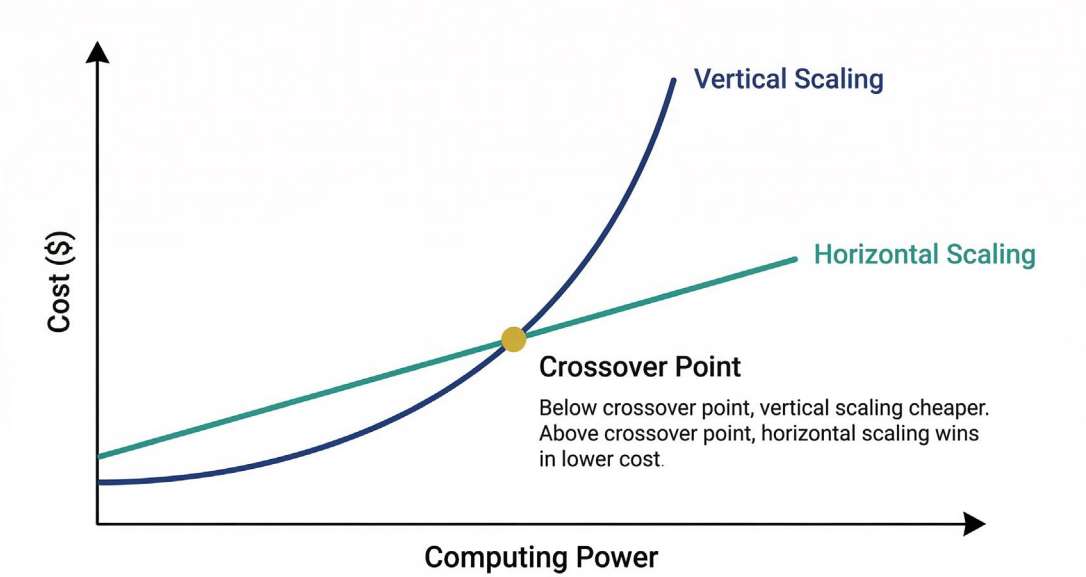
Wall #3: The Cost Wall

Vertical Scaling Cost Curve

- 2× the power ≠ 2× the cost
- More like 4–10× the cost
- Premium hardware, vendor lock-in
- Supercomputers: tens of millions $

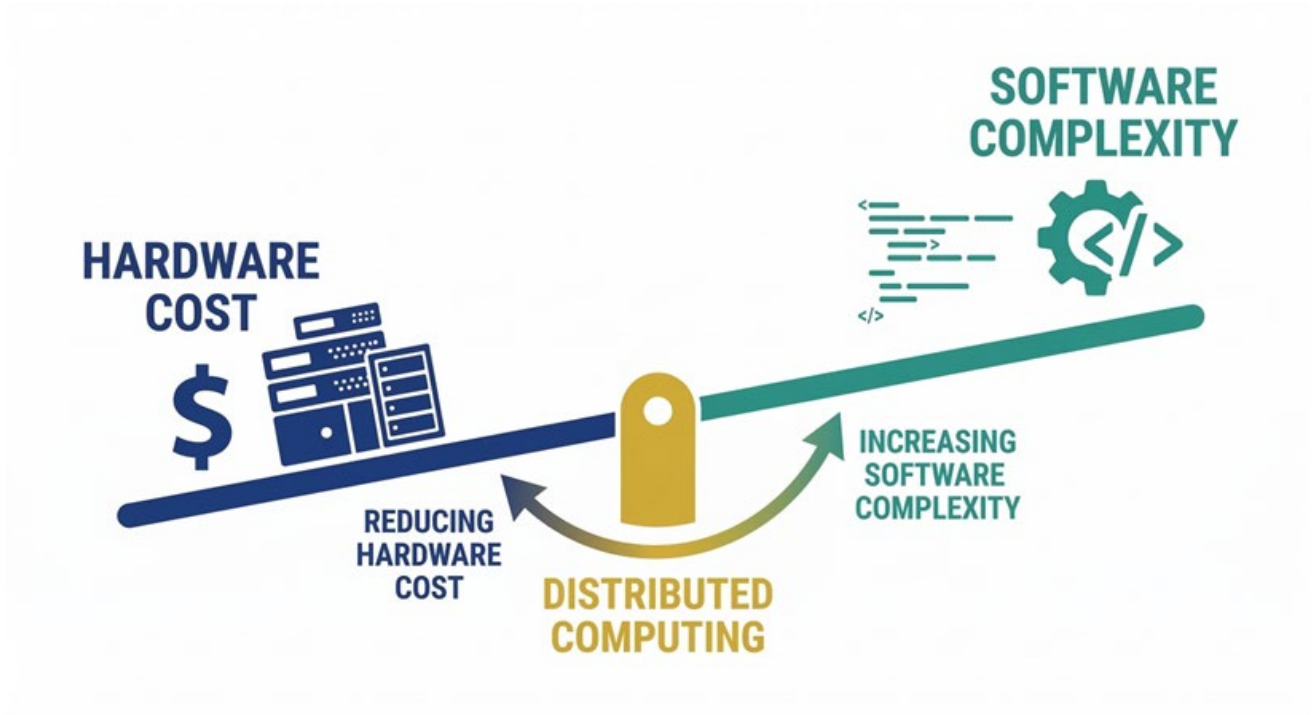
Horizontal Scaling Cost Curve

- Commodity PCs: hundreds $ each
- Replaceable, interchangeable
- Buy in bulk, discard on failure



Google's insight (late 1990s): it is cheaper to buy 1,000 commodity PCs and deal with coordination complexity than to buy one supercomputer.

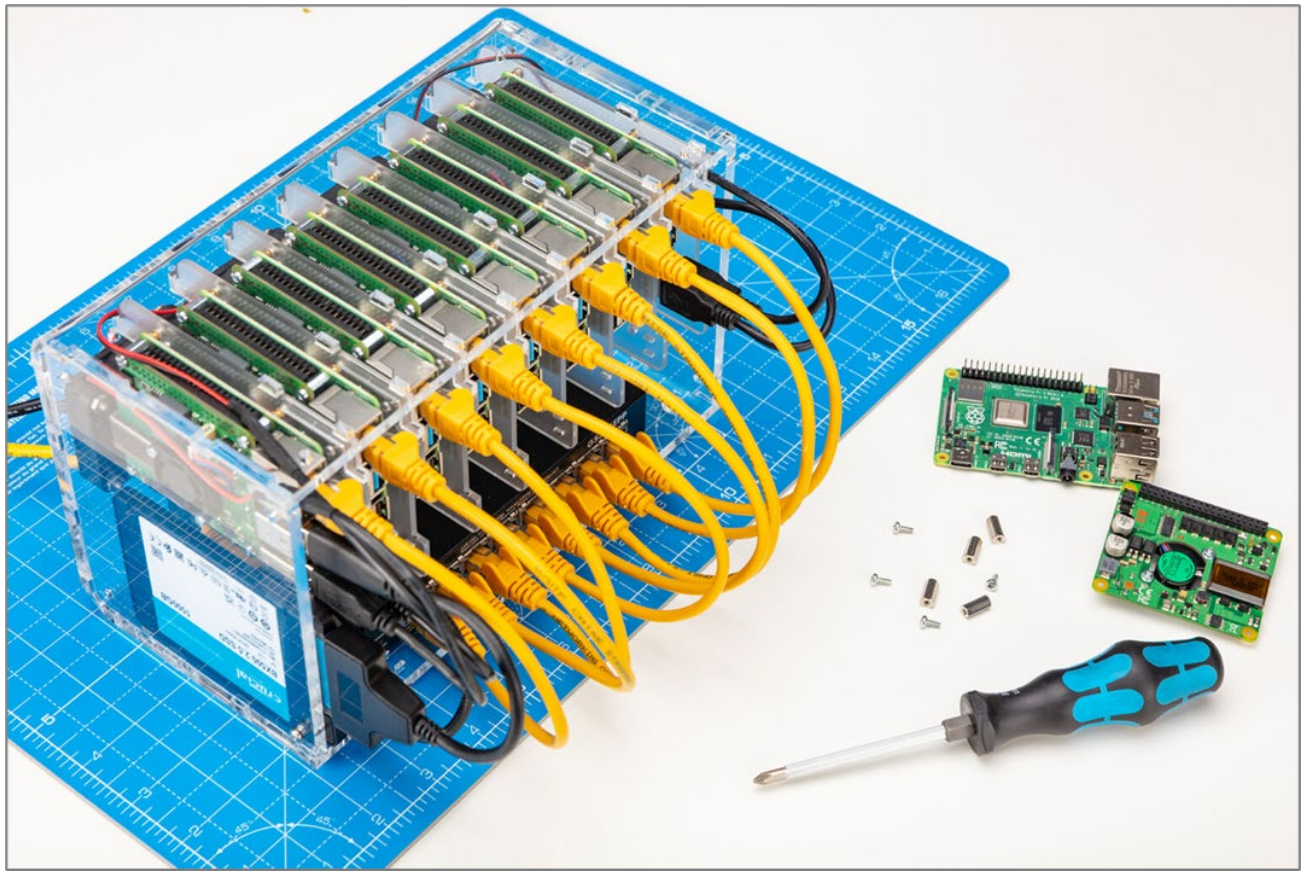
The Key Mental Model

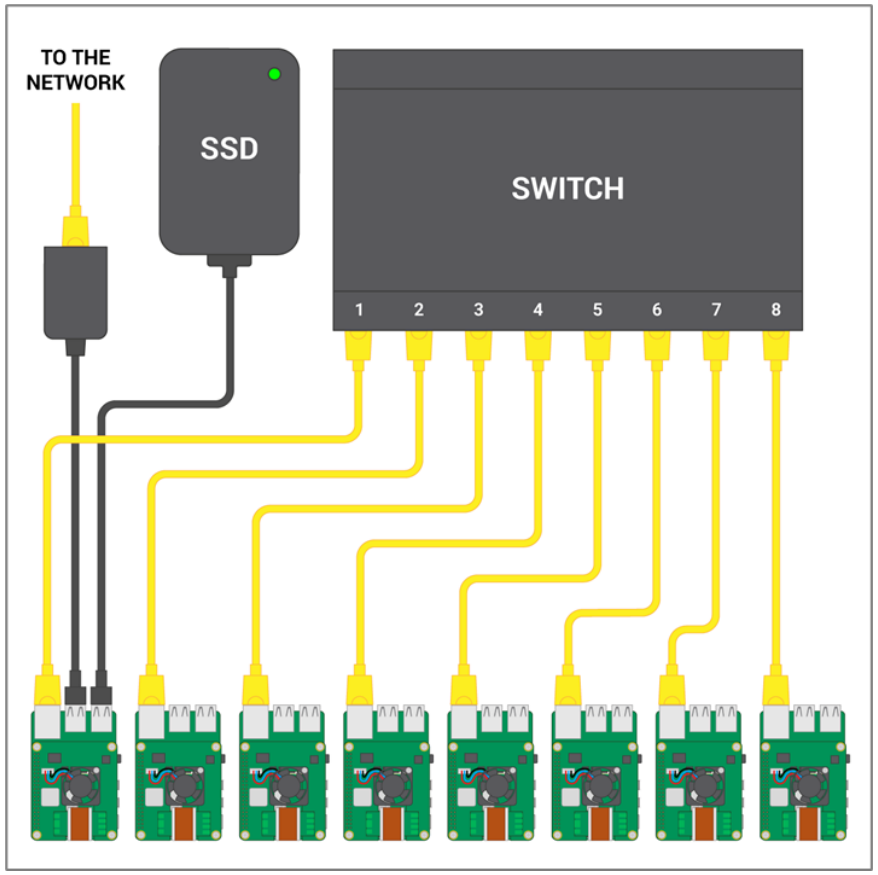


Everything we study for the rest of the semester is about managing the software complexity that horizontal scaling introduced.

This is also why cloud computing exists: AWS, GCP, and Azure absorbed the software complexity so individual organizations don't have to build it themselves.

DIY Example: The Raspberry Pi Cluster





Yes, this can run Hadoop / Spark.

The concept is the same whether it's 8 Raspberry Pis on a desk or 10,000 servers in a warehouse.

What Is a Distributed System?

Multiple computers working together to solve a common problem, appearing to the end user as a single system.

No shared memory
Each machine has its own RAM

Scalability
Can add machines to handle more work

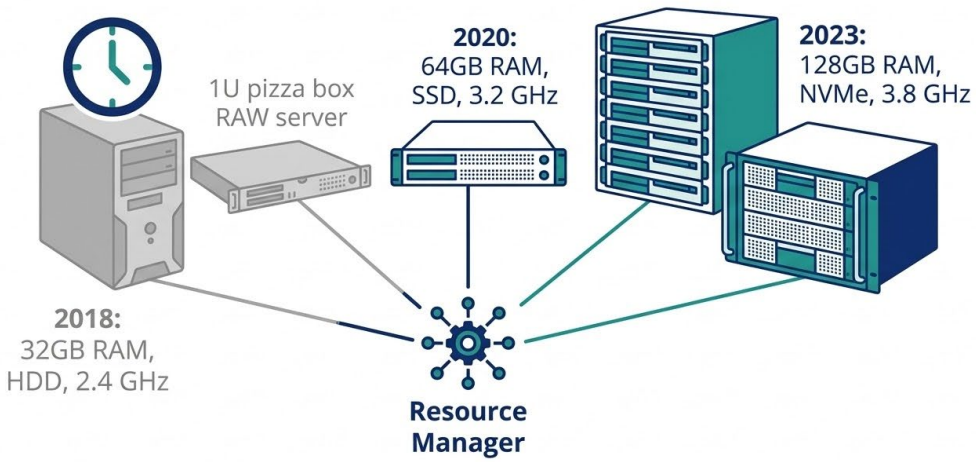
No shared clock
'Which happened first?' is genuinely hard

Fault tolerance required
Machines will fail; system must keep working

Parallel computing
Work happens simultaneously

Heterogeneity possible
Different hardware, OS, capabilities

Heterogeneity: Not All Machines Are Equal



- A real cluster isn't 1,000 identical machines
- Some nodes compute faster, some have more memory
- Equal-sized chunks won't finish evenly — the "straggler problem"
- Resource manager (YARN, Spark) tracks each node's capabilities
- Even in cloud, "noisy neighbors" cause variation

Supercomputer vs. Cluster Computing

Supercomputer (HPC)

- Purpose-built, tightly coupled
- Identical processors in lockstep
- Shared memory architecture
- Astronomically expensive
- Used for: weather, nuclear sims, physics

Cray J90 (1995): High performance supercomputer

Cluster Computing

- Commodity hardware, loosely coupled
- Heterogeneous machines
- No shared memory — network comms
- Cheap, but machines will fail
- Used for: web indexing, analytics, ML

Beowulf clusters (1994): NASA proved cheap hardware + clever software could rival specialized machines.

When Do You Actually Need This?

You probably need it when...

- Data exceeds one machine's memory or storage
- Computation takes hours/days on one machine
- Need to join datasets across billions of rows
- Fault tolerance across long-running jobs
- Process streaming data continuously

You probably don't need it when...

- Data fits in memory (64–256 GB is a lot)
- Each transaction touches a small slice
- A well-tuned SQL database handles the load
- Analysis can run overnight and that's fast enough

Not every big dataset needs a distributed system. Identify the binding constraint first.

Three Walls, One Response

Physics
Clock speeds plateaued

Data
Volumes exploded

Economics
Vertical scaling too expensive

Spread computation across many cheap machines

The rest of this lecture: How did they actually do it? What problems did they have to solve?

Questions So Far?

Before we move to the history of how these problems were solved...

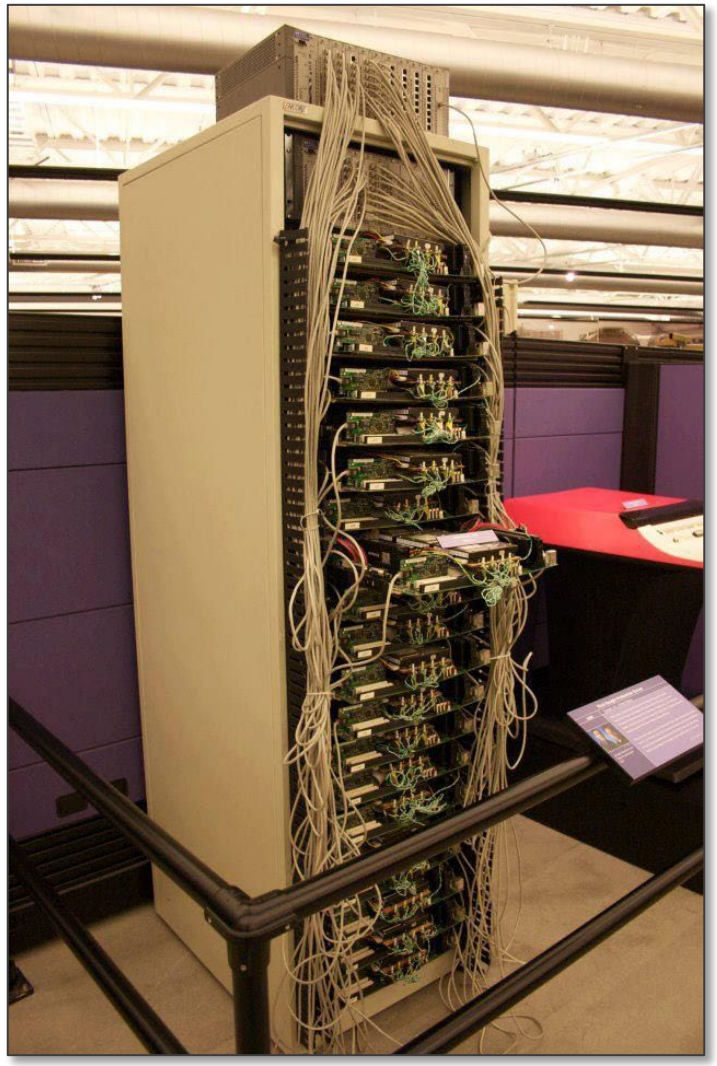
A Brief History of Making Computers Cooperate

Part 2 · From Mainframes to MapReduce to the Modern Stack

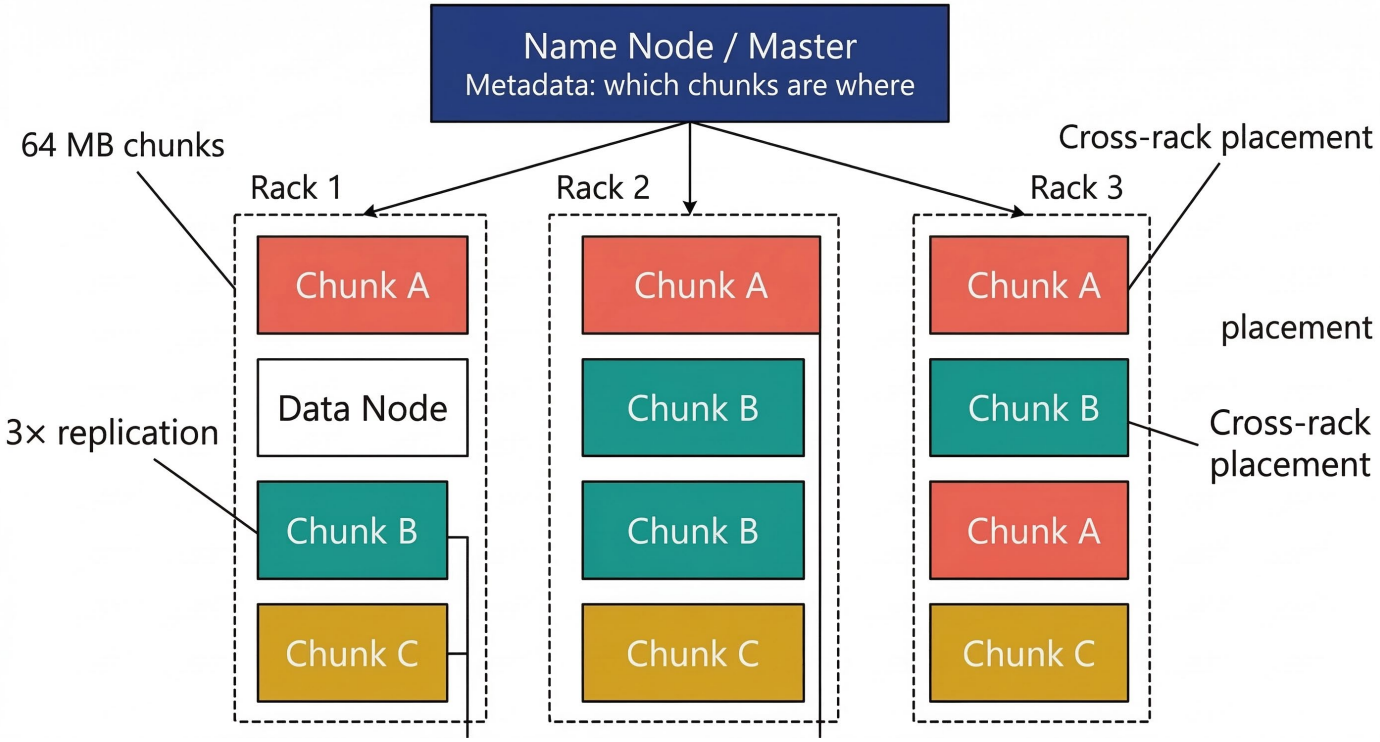
Google's Problem (Late 1990s)

1. Store a copy of the entire web (petabytes, growing fast)
    I. 20+ billion web pages × 20KB = 400+ TB
    II. ~1,000 hard drives just to store it
2. Build an index of every word on every page
3. Rank results using PageRank (massive iterative graph computation)
4. Do all of this cheaply, reliably, on hardware that will fail
    I. 1 computer reads 30–35 MB/sec from disk
    II. That's ~4 months to even *read* the web, before doing anything useful
5. **No existing system could do this. They had to invent new infrastructure.**

No existing system could do this. They had to invent new infrastructure.



The Google File System (GFS), 2003



Files are enormous, rarely updated, read sequentially

Hardware failure is a normal operating condition

Replicas on different racks — rack failure can't destroy all

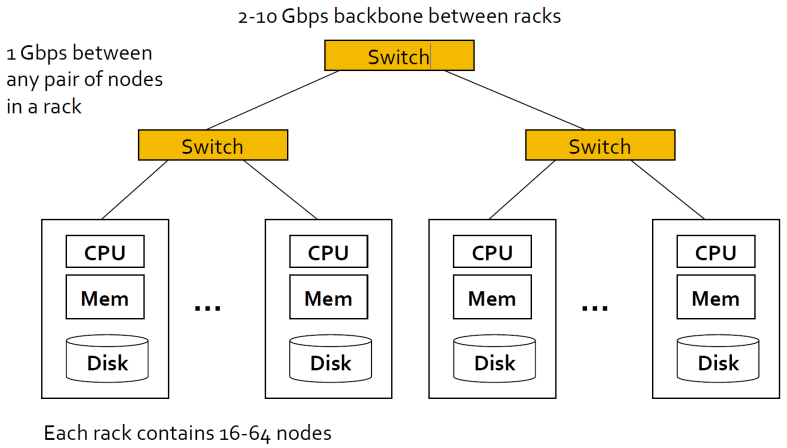
Failure at Scale: The Birthday Paradox

Birthday Paradox

- 23 people in a room
- $P = 365/365 \times 364/365 \times \ldots$
- = ~49.3% no shared birthday
- **More likely than not!**

Cluster Failure

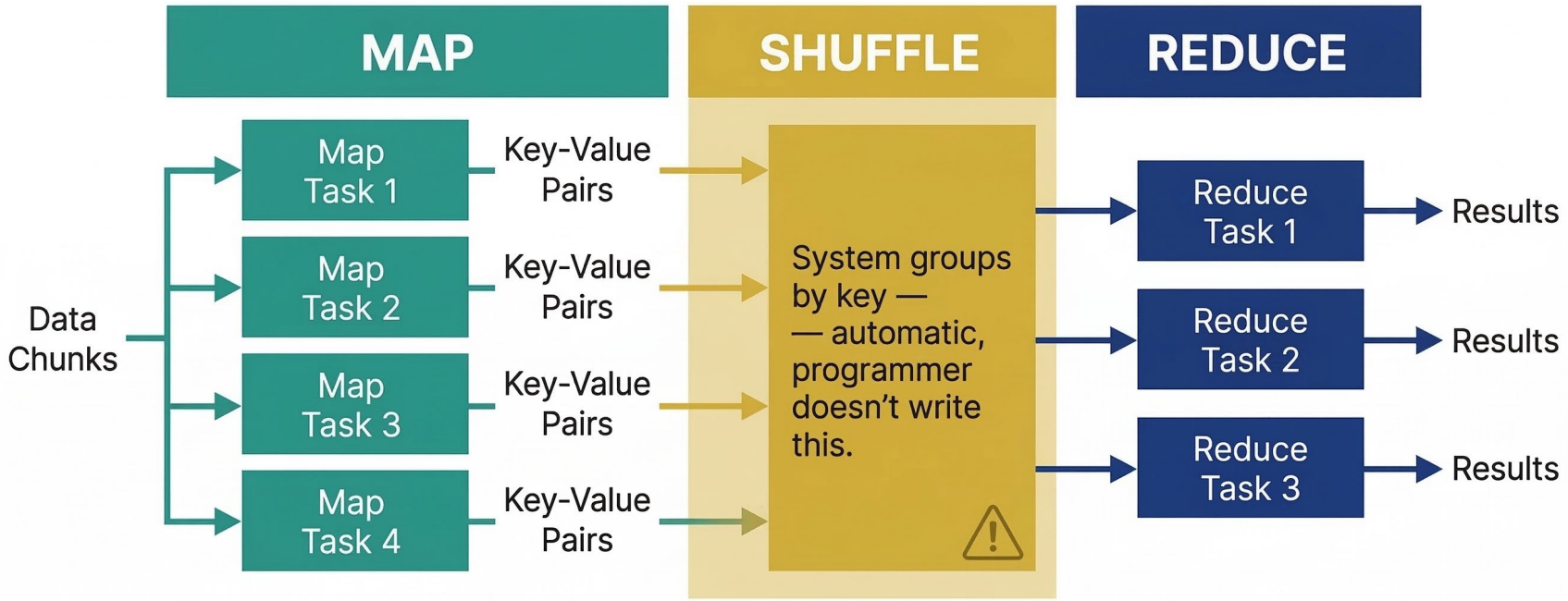
- 1,000 machines, each 99.9% reliable/hr
- $P(\text{all survive}) = 0.999^{1000}$
- = ~36.8%
- **You lose ≥1 machine most hours**



Same structure: multiply independent 'nothing bad happens' probabilities. Cumulative risk is far higher than intuition suggests.

Fault tolerance isn't a feature. It's a prerequisite.

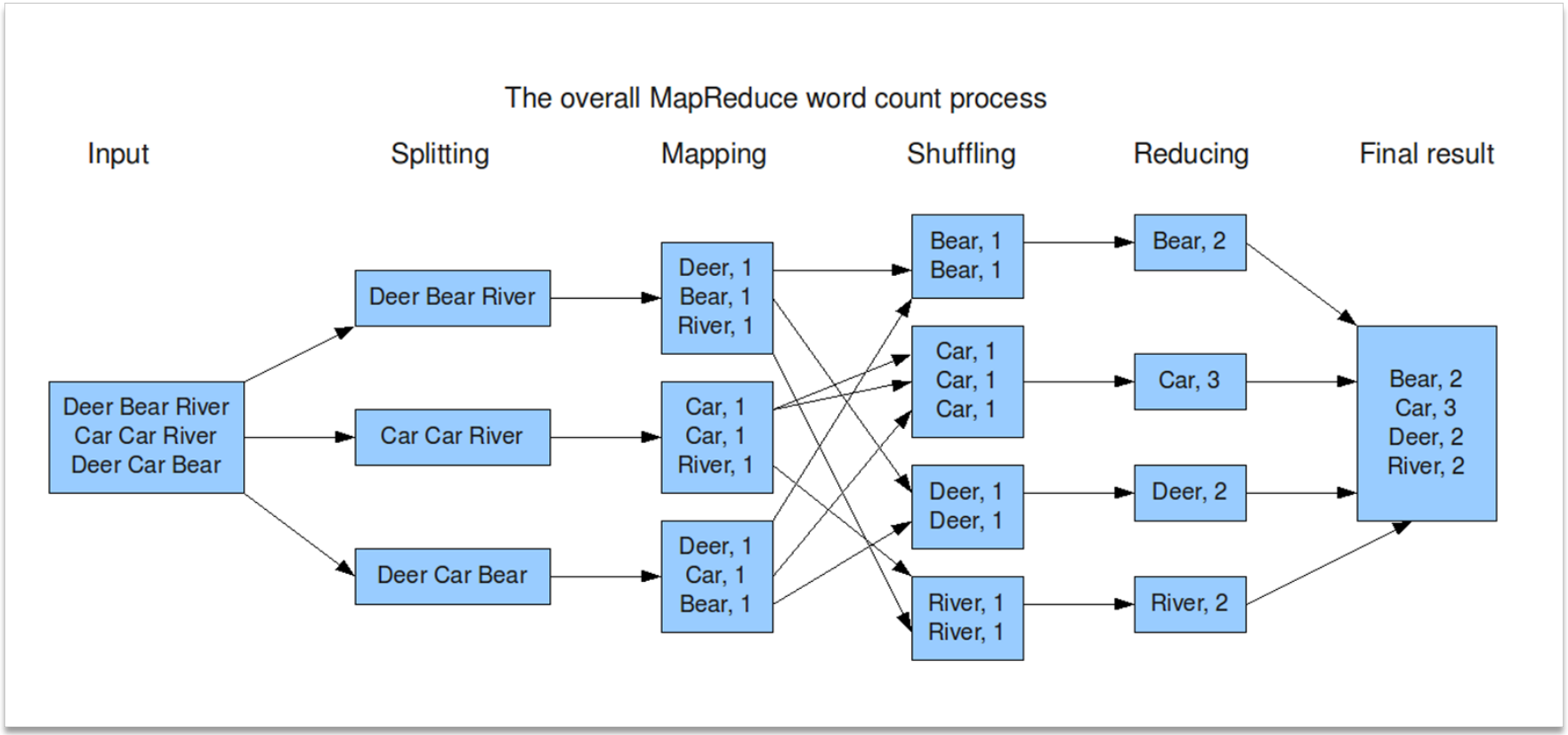
MapReduce (2004): The Breakthrough



The programmer writes two functions: Map and Reduce.

The system handles everything else: scheduling, fault tolerance, data movement.

MapReduce: Word Count Example



Embarrassingly Parallel vs. Coordination-Required

Embarrassingly Parallel (Map)

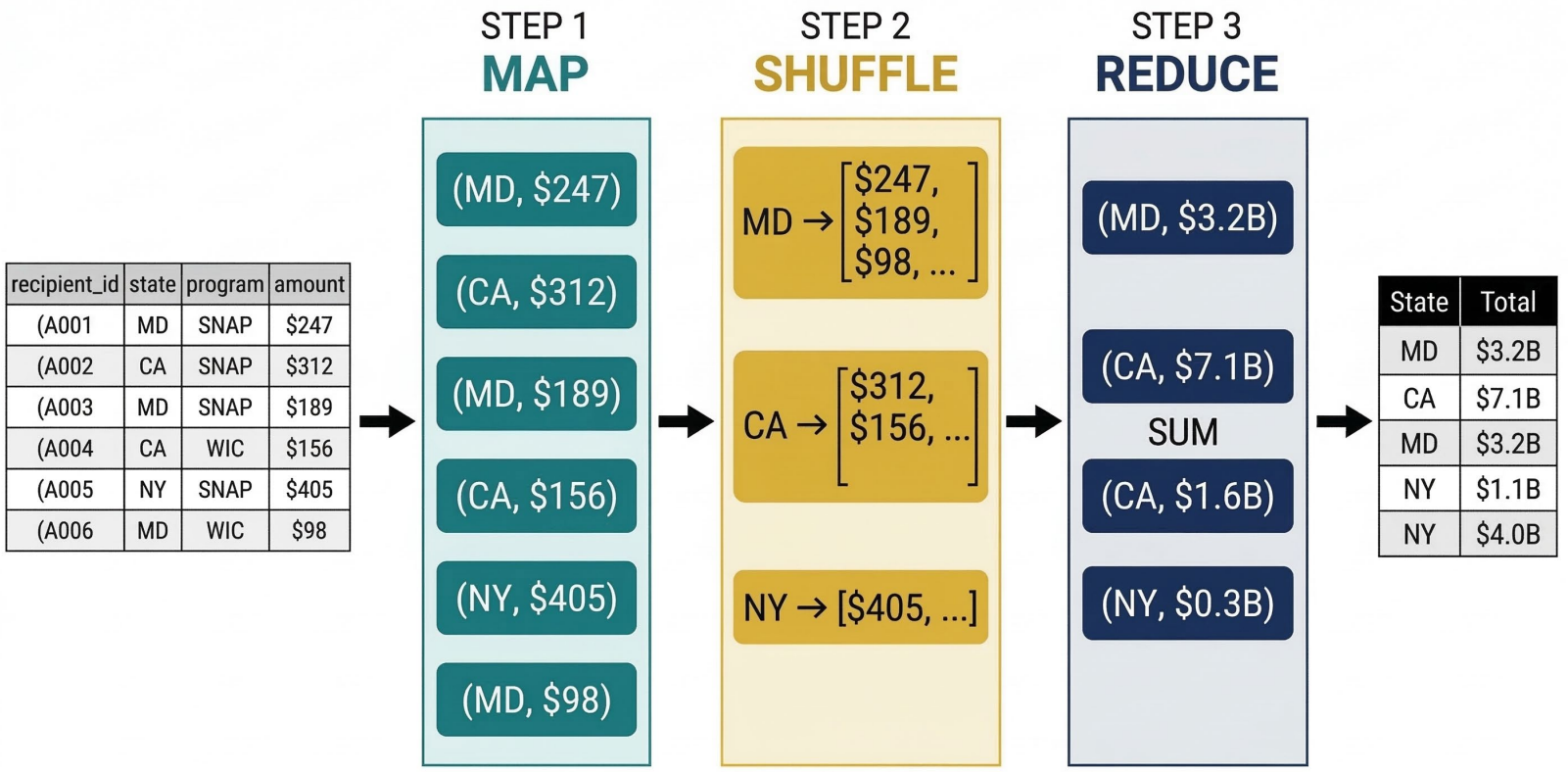
- Each work item is completely independent
- No item depends on any other
- Zero coordination needed
- Example: filter 10M records independently

Requires Coordination (Reduce)

- Output depends on combining data from multiple sources
- Data must be brought together — the shuffle
- Bottleneck: network data movement
- Example: sum all payments by state

Map is easy (and fast). Reduce is hard (and expensive). The shuffle bridges the gap.

Example: Total SNAP Disbursements by State

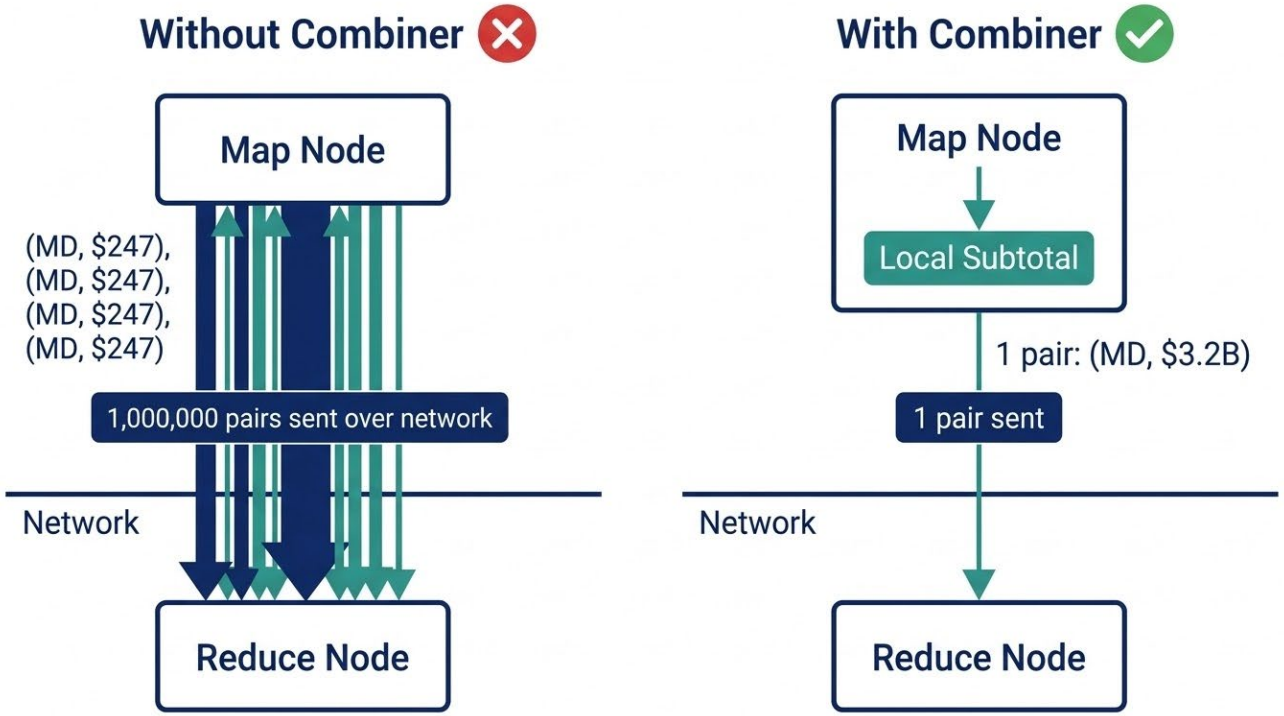


This is GROUP BY state, SUM(amount) — the same SQL you wrote in Weeks 2–3. MapReduce is how it runs across 1,000 machines.

The Power of Choosing Your Key

Same data, different questions:

- Key = (state, program) → totals by state AND progr
- Key = (month, state) → monthly trends by state
- Key = recipient_id → total per recipient (fraud dete



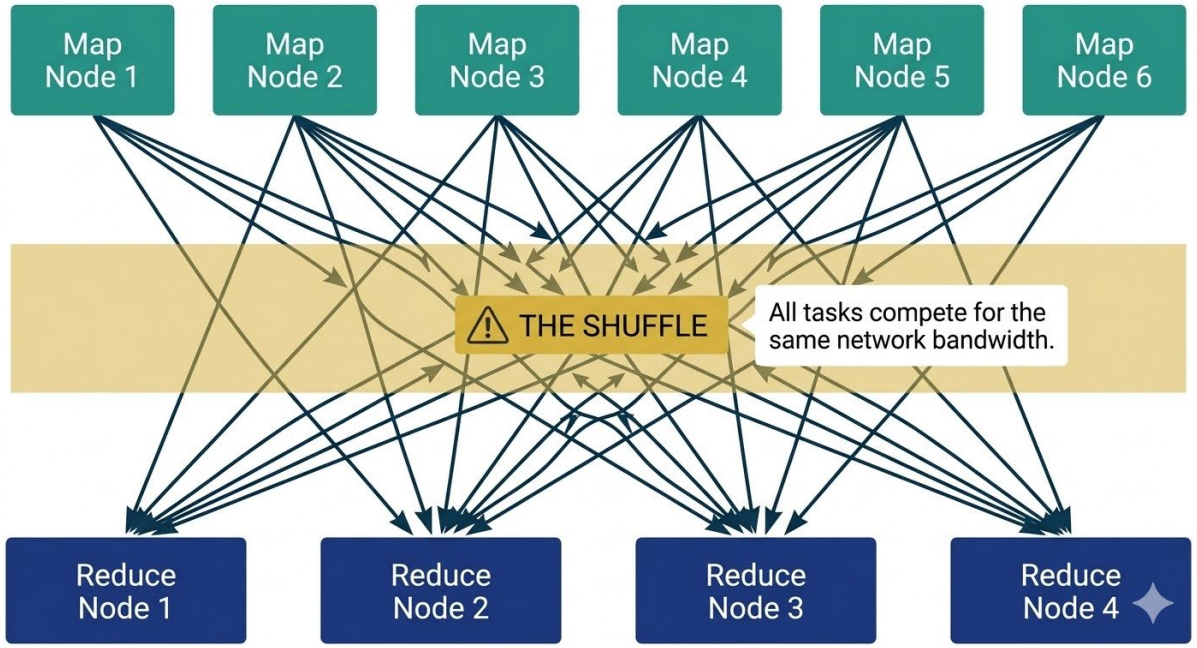
When Reduce is associative and commutative (e.g., addition), do partial aggregation at the Map node before shipping across the network.

MapReduce Implements SQL

SQL Operation	Map Step	Reduce Step
WHERE (filter)	Apply condition, emit qualifying records	Identity (pass through)
GROUP BY...SUM	Emit (grouping_key, value)	Sum values per key
JOIN	Emit (join_key, tagged record)	Match records with same key
SELECT columns	Emit only desired columns	Eliminate duplicates
COUNT DISTINCT	Emit (key, item)	Count unique items per key

Every distributed SQL engine (Spark SQL, Presto, BigQuery, Snowflake) implements these patterns.

Why Joins Are Expensive



· All records with the same join key must end up on the same machine

· The network is shared: when hundreds of tasks shuffle simultaneously, they all compete

· Skew: one key with far more records becomes a bottleneck

Practical rule: minimize data movement. This is the primary optimization strategy.

How MapReduce Handles Failure

Master fails: Entire job restarts. The Master is a single point of failure. In practice, replicated with standby nodes.

Map worker fails: All its tasks redone — even completed ones — because output was on local disk, now inaccessible.

Reduce worker fails: Only currently executing tasks rescheduled. Completed output already in DFS (replicated).

The blocking property: Tasks deliver output only after completion. A failed task hasn't contaminated downstream computation.

The asymmetry is a direct consequence of where intermediate data is stored (Map output = local disk; Reduce output = DFS).

Hadoop: The Open-Source Echo (2006–2014)

- **2006:** Doug Cutting & Mike Cafarella release HDFS + MapReduce (from Nutch web crawler)
- **2008–2012:** Explosive adoption — Yahoo, Facebook, LinkedIn

Key Ecosystem Components:

- **HDFS:** Distributed file system (storage layer)
- **Hive:** SQL interface on MapReduce — ancestor of Spark SQL
- **YARN:** Separated resource mgmt from MapReduce — enabled Spark on Hadoop clusters
- Pig, Oozie, etc.: Existed. Effectively dead in 2026.

Hadoop's Limitations

- ✗ Everything written to disk between stages. Painfully slow for iterative algorithms (ML, graph processing).
- ✗ Multi-step jobs = chaining MapReduce jobs. Fragile, hard to debug.
- ✗ Batch only. Submit a job, wait, get results. No interactivity.
- ✗ Writing raw Map/Reduce functions was tedious for common operations.

These limitations motivated a Berkeley PhD student named Matei Zaharia to ask a simple question...

The Spark Insight (2009–2010)

What if we kept data in memory?

Lazy Evaluation

Spark doesn't execute transformations until an action forces it. Intermediate results may never be written to disk — created and consumed at the same compute node.

Lineage-Based Fault Tolerance

Instead of replicating intermediate data (expensive), Spark remembers the recipe. If a partition is lost, retrace the recipe and recompute just what was lost.

What about data bigger than memory? Spark spills to disk when memory is exhausted. Slower, but still works.

We go deep on Spark in Weeks 10–11. Today: just understand why these innovations matter.

MapReduce in 2026

As a direct programming model:

Effectively dead

- Nobody writes raw Map/Reduce
- Most clusters migrated to Spark/cloud

As a set of ideas:

Permanently embedded

- Spark = generalization of MapReduce
- Every SQL engine uses these patterns
- The shuffle is still the shuffle
- Combiners → Spark's reduceByKey

I teach MapReduce because it's the simplest possible distributed programming model, and therefore the right place to learn the concepts.

The Hard Problems

Part 3 · What Makes Distributed Computing Actually Difficult

The Dominant Cost: Moving Data

Computation is simple: The algorithm at each task is usually linear in input size. Processing each record takes nanoseconds.

Network is slow (relatively): Network bandwidth seems fast but is slow compared to how fast a CPU processes data.

Network is shared (the key insight): CPU and memory are per-node resources. The network is shared. When hundreds of tasks shuffle simultaneously, they compete for the same interconnect bandwidth.

Practical rule: when designing distributed computations, minimize data movement.

Data Partitioning: Where Does the Data Live?

10 TB of data, 100 machines. How do you split it?

Hash Partitioning

- Hash function assigns records to machines
- ✅ Even distribution
- ❌ Destroys locality — "MD" records scattered

Range Partitioning

- Assign key ranges (A–M → Machine 1)
- ✅ Related data stays together
- ❌ Hotspots if distribution is uneven

Every query depends on where data physically sits. A bad partitioning strategy turns a 10-minute job into a 10-hour job.

Fault Tolerance: Three Strategies

Replication (HDFS)

Store 3× copies
Simple recovery
Expensive in storage
Analogy: hot standby

Checkpointing

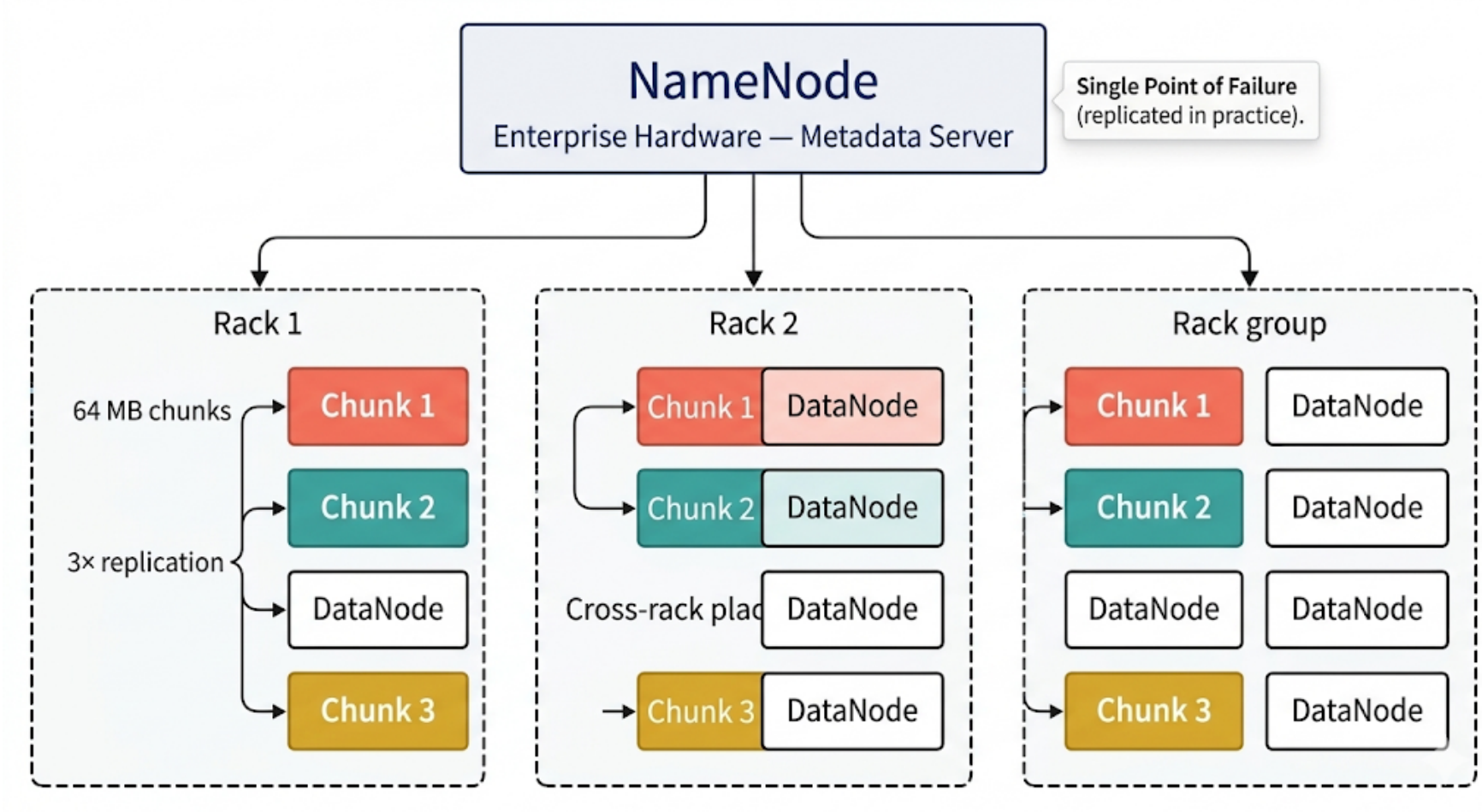
Periodically save state
Restart from checkpoint
Wastes work since last save
Analogy: periodic backup

Lineage (Spark)

Remember the recipe
Recompute just what's lost
Free during normal ops
Analogy: cold backup

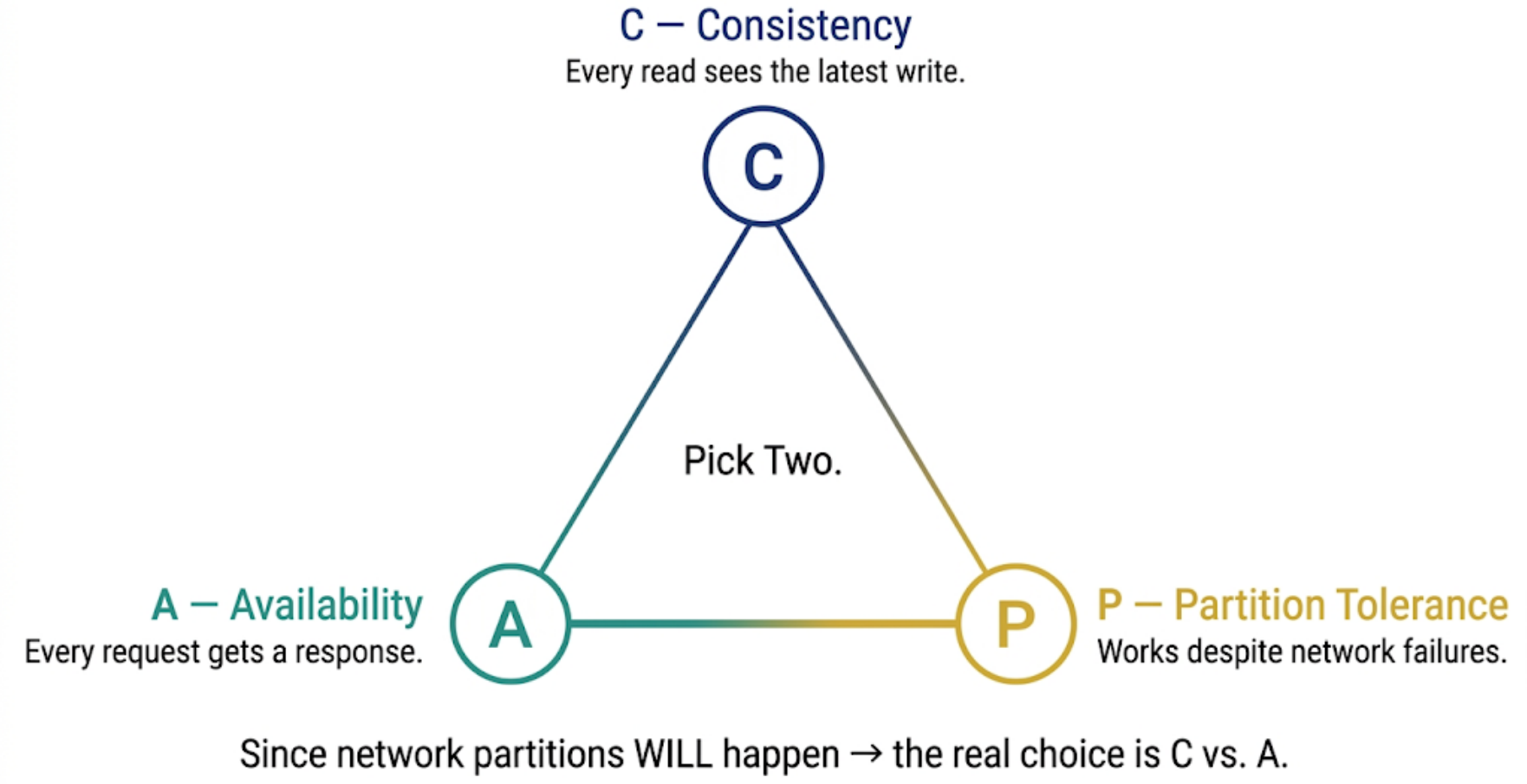
No single best strategy. Each involves a tradeoff between storage cost, recovery time, and normal-operation speed.

HDFS: How It Actually Works



HDFS is not the dominant storage in 2026 (S3 replaced it), but it's the clearest example of distributed storage.

The CAP Theorem



Brewer (2000), formally proved by Gilbert & Lynch (2002)

CAP in Practice

Choose Consistency: Your bank's transaction system. You'd rather see "system unavailable" than a wrong balance.

Choose Availability: Social media feeds. Showing a slightly stale feed is better than showing nothing. "Eventual consistency."

Tools You Know:

- Snowflake, Redshift → lean toward consistency
- S3 → eventual consistency for overwrites
- DynamoDB, Cassandra → configurable, lean availability

Coordination: The Overhead Tax

- Distributed systems need coordination: who processes what? In what order? When to commit?
- Coordination requires communication, and communication is slow
- Leader election: How do nodes agree on who's in charge? What if the leader fails?
- Consensus protocols (Paxos, Raft): famously hard to implement correctly

Key insight: Coordination overhead is why you can't just throw more machines at every problem. Diminishing returns exist — the ceiling is just much higher than vertical scaling.

An Apparent Contradiction

Principle 1: Data Locality
Move computation to where the data lives. Minimize network traffic.

Principle 2: Decoupled Storage
Separate storage from compute. Shut down compute when not in use.

The Resolution: Modern architectures compensate with high-bandwidth connections, smart caching, columnar formats (Parquet/ORC), and partition pruning. You trade data locality for elasticity and cost savings.

It's a tradeoff, not a contradiction — understanding this helps you evaluate architectural choices.

The Hard Problems at a Glance

Problem	What It Is	Why It's Hard	Where You'll See It
Partitioning	Where does data live?	Determines every operation's perf	Databricks partition columns
Shuffle	Moving data between machines	Shared network contention	Spark job optimization
Fault Tolerance	Machines will fail	Must operate during failures	HDFS replication, Spark lineage
Consistency	Can everyone agree?	Can't have C + A + P	S3 eventual consistency
Coordination	Who's in charge?	Comm overhead limits scaling	YARN, cluster managers

Every tool you'll use — Spark, Databricks, EMR — solves these same problems with different tradeoffs.

Where You've Already Seen This

Part 4 · You've been using distributed systems all semester

Snowflake (Weeks 5–6): A Distributed System

Virtual warehouses → clusters of compute nodes
Resizing (S → M → L) → adding/removing nodes
Micro-partitioning → automatic data partitioning strategy
Storage-compute separation → decoupled architecture
Query execution → SQL compiled into distributed execution plans

You've been using distributed computing since Week 5.

AWS Services You've Used (Week 7)

AWS Glue

- Runs Spark under the hood
- Your ETL jobs were distributed computations
- S3 → Glue → Athena = distributed storage + compute + query

Amazon S3

- A distributed storage system with replication and eventual consistency
- "Eleven 9s" of durability (99.999999999%) = distributed replication across facilities
- When a vendor quotes durability, they're describing their replication strategy

The Modern Ecosystem (Orienting, Not Exhaustive)

- **General-purpose compute:** Spark (Weeks 10–11)
- **Managed platforms:** Databricks (Week 10), EMR (Week 12)
- **Distributed SQL:** Spark SQL, Presto/Trino, BigQuery, Redshift Spectrum, Snowflake
- **Streaming:** Kafka, Flink (mention only)
- **Python-native:** Dask, Ray (mention only)

All compile SQL/computation into distributed execution plans. They differ in storage management, compute allocation, and operational complexity.

The concepts from today are universal. The tools are implementations.

Why This Matters for Policy

Administrative Data Linkage

Joining IRS + CMS + SSA across billions of records

Real-Time Dashboards

COVID tracking, benefits monitoring, emergency response

Census Processing

Tabulation + differential privacy + publication deadline

Satellite/Geospatial

Remote sensing for environmental policy — continental scale

These tools aren't Silicon Valley luxuries. They're operational necessities for organizations serving hundreds of millions of people.

Questions Before We Look Ahead?

The Mental Model to Carry Forward

Distributed computing is about tradeoffs, not magic:

- Cost vs. complexity · Consistency vs. availability
- Latency vs. throughput · Disk vs. memory (Hadoop → Spark)
- Replication vs. lineage (fault tolerance)

When evaluating any tool — ask:

- How does it partition data?
- How does it handle failure?
- What consistency guarantees does it make?
- Where does it sit on the cost/complexity curve?